



ΕΡΓΑΣΤΗΡΙΟ 2

Επεξεργασία συμβολοσειρών (strings)

Ας θεωρήσουμε ότι στο όνομα μεταβλητής `string1` έχουμε μία συμβολοσειρά (string). Τότε μπορούμε να χρησιμοποιήσουμε τις ακόλουθες εντολές.

- `string1[i]`: λαμβάνουμε το *i*-οστό στοιχείο συμβολοσειράς. Το *i* μπορεί να είναι είτε το 0 (πρώτο στοιχείο της συμβολοσειράς) ή θετικός αριθμός ή αρνητικός αριθμός (με -1 μπορούμε να προσπελάσουμε την τελευταία θέση, -2 η προτελευταία, κ.ο.κ.).
- `string1[i:j]`: εξαγωγή υπό-συμβολοσειράς από το χαρακτήρα στη θέση *i* μέχρι το χαρακτήρα στη θέση *j*-1.
- `string1 + string2`: συνένωση συμβολοσειρών και δημιουργία νέας χωρίς να επηρεάζονται οι συμβολοσειρές `string1` και `string2`.
- `n*string1`: επαναλαμβανόμενη συμβολοσειράς *n* φορές (το *n* πρέπει να είναι ακέραιος).
- `s in string1`: ελέγχει αν το *s* ανήκει στην `string1` (boolean).
- `s not in string1`: ελέγχει αν το *s* δεν ανήκει στην `string1` (boolean).
- `for x in string1`: διατρέχει τα στοιχεία συμβολοσειράς.

Επίσης, έχουμε τη συνάρτηση `len()` η οποία παίρνει σαν είσοδο (όρισμα) μία συμβολοσειρά και επιστρέφει τον αριθμό των στοιχείων συμβολοσειράς (μήκος συμβολοσειράς).

Σε μία οποιοδήποτε συμβολοσειρά μπορούμε να εφαρμόσουμε μεθόδους.

- `string1.index(s)`: η θέση της πρώτης εμφάνισης του στοιχείου *s* στην συμβολοσειρά. Επιστρέφει ακέραιο αριθμό.
- `string1.count(s)`: αριθμός εμφανίσεων του στοιχείου *s* στην συμβολοσειρά. Επιστρέφει ακέραιο αριθμό.
- `string1.lower()`, `string1.upper()`: μετατρέπει όλους τους χαρακτήρες σε πεζούς και σε κεφαλαίους αντίστοιχα. Επιστρέφουν μια νέα συμβολοσειρά, χωρίς να επηρεάζεται η συμβολοσειρά `string1`.
- `string1.capitalize()`: μετατρέπει σε κεφαλαίο τον πρώτο χαρακτήρα της συμβολοσειράς (αν αυτός είναι γράμμα). Επιστρέφει μια νέα συμβολοσειρά, χωρίς να επηρεάζεται η συμβολοσειρά `string1`.
- `string1.replace(old, new)`: αντικαθιστά στα περιεχόμενα της `string1` την συμβολοσειρά `old` με την συμβολοσειρά `new`. Επιστρέφει μια νέα συμβολοσειρά με τις αλλαγές, χωρίς να επηρεάζεται η συμβολοσειρά `string1`.

Επεξεργασία των strings με βρόγχο for:

Μπορείτε να επεξεργαστείτε ένα string μέσω for loop. Σκεφτείτε το string σαν μια ακολουθία από χαρακτήρες. Με παρόμοιο τρόπο όπως χρησιμοποιούσαμε τον βρόγχο for σε μια ακολουθία αριθμών (η οποία δημιουργείται με την εντολή range), μπορούμε να χρησιμοποιήσουμε την εντολή for για να «περάσουμε» από κάθε χαρακτήρα μέσα σε μια συμβολοσειρά.



Έστω πως θέλουμε να μετρήσουμε το πλήθος των κεφαλαίων και μικρών γραμμάτων ξεχωριστά σε ένα string.

```
uppercase = 0
lowercase = 0

for char in "This is an example with strings!":
    if char.isupper():
        uppercase += 1

    if char.islower():
        lowercase += 1

print ('Total Characters: ', len("This is an example with strings!"))
print ('Total Uppercase Characters: ', uppercase)
print ('Total Lowercase Characters: ', lowercase)
```

Result:

```
Total Characters: 32
Total Uppercase Characters: 1
Total Lowercase Characters: 25
```

Οι συναρτήσεις `.isupper()` (για κεφαλαία) και `.islower()` (για μικρά) καλούνται πάνω σε ένα χαρακτήρα και επιστρέφουν `true` σε περίπτωση που το γράμμα είναι κεφαλαίο ή μικρό αντίστοιχα. Μπορούν να καλεστούν και πάνω σε συμβολοσειρά και επιστρέφουν `True` αν όλοι οι χαρακτήρες της συμβολοσειράς είναι κεφαλαίοι ή πεζοί (μικροί) αντίστοιχα, αλλιώς `False`.

Δείτε πιο κάτω κάποια παραδείγματα με strings:

Παράδειγμα 1

```
s = "I like python"
print("Length:", len(s), "characters") # Length: 13 characters
print(s[2]) # τυπώνει το τρίτο γράμμα του string s
print(s[-1]) # τυπώνει το τελευταίο γράμμα του string s
s1 = s[2:5] # εξάγουμε την υπο-συμβολοσειρά από τη θέση 2 έως # και τη
            # θέση 4 που φαίνεται στην πιο κάτω γραμμή
print(s1) # lik
print(s.index("y")) # η θέση του χαρακτήρα y μέσα στο s είναι 8
s2 = s.replace("like", "love")
print(s2) # I love python
print(s) # I like python
s3 = s2.upper()
print(s3) # I LOVE PYTHON
```

Για συμβολοσειρές, έχουμε επίσης την εξής μέθοδο.

- `string1.split(d)`: τεμαχίζει την `string1` όπου υπάρχει το στοιχείο `d` και παράγει μία λίστα με το αποτέλεσμα (με τις υπό-συμβολοσειρές που δημιουργούνται).



Παράδειγμα 2

Θα διαβάσουμε σειρά αριθμών από το πληκτρολόγιο και θα τους τυπώσουμε σαν λίστα.

```
line = input("Give numbers separated by comma: ")
strList = line.split(',')
print(strList)
```

Αποτέλεσμα εκτέλεσης (με πράσινο η είσοδος του χρήστη)

```
Give numbers separated by comma: 1.2,2.3,3.14,6
['1.2', '2.3', '3.14', '6']
```

Παράδειγμα 3

Διαβάστε μία έκφραση η οποία παριστάνει ένα άθροισμα (χωρίς ενδιάμεσα κενά), και τυπώστε τη λίστα των αριθμών χωρίς τον χαρακτήρα πρόσθεσης +.

```
s = input("Give an expression for a sum: ")
L = s.split("+")
print(L)
```

Αποτέλεσμα εκτέλεσης (με πράσινο η είσοδος του χρήστη)

```
Give an expression for a sum: 1.2+4.5+6.4+14.8
['1.2', '4.5', '6.4', '14.8']
```

Για να βρούμε την θέση στοιχείου σε συμβολοσειρά έχουμε τις εξής μεθόδους

- `s.index(d)`: θέση του στοιχείου `d` στην `s`.
- `s.find(d)`: θέση του στοιχείου `d` στην `s`.

Παράδειγμα 4

Ας δούμε τις δύο διαθέσιμες μεθόδους για εύρεση θέσης στοιχείου σε συμβολοσειρά.

```
s = "abcdefghijk"
print(s.find("c")) # θα τυπώσει 2
print(s.index("c")) # θα τυπώσει 2
```

Οι δύο αυτές μέθοδοι όμως συμπεριφέρονται διαφορετικά όταν το ζητούμενο στοιχείο δεν υπάρχει στην συμβολοσειρά. Υποθέστε ότι οι 2 πιο κάτω γραμμές είναι συνέχεια του ακριβώς πιο πάνω παραδείγματος.

```
print(s.find("z")) # θα τυπώσει -1
```



```
print(s.index("z"))
Traceback (most recent call last):
  File "", line 5, in
ValueError: substring not found
```

Έχουμε επίσης τις μεθόδους

- `s.isalpha()`: ελέγχει αν όλοι οι χαρακτήρες σε μία συμβολοσειρά είναι γράμματα.
- `s.isdigit()`: ελέγχει αν όλοι οι χαρακτήρες σε μία συμβολοσειρά είναι αριθμητικά ψηφία.

Επίσης τις μεθόδους (χρήσιμες για το formatting των strings)

- `s.strip()`: αφαιρεί τους κενούς χαρακτήρες από την αρχή και το τέλος της `s`.
- `s.lstrip()`, `s.rstrip()`: αφαιρεί τους κενούς χαρακτήρες από την αρχή (αριστερά) ή από το τέλος (δεξιά) της `s` αντίστοιχα.

Παράδειγμα 5

Διαβάστε μια συμβολοσειρά από το χρήστη και μετρείστε το πλήθος των γραμμάτων του αγγλικού αλφαβήτου και το πλήθος των αριθμητικών χαρακτήρων που περιέχει.

```
sentence = input("Give a sentence: ")
countNum = 0
countLetters = 0
for c in sentence:
    if c.isdigit():
        countNum = countNum + 1
    if c.isalpha():
        countLetters = countLetters + 1

print(f"We have {countNum} numbers and {countLetters} letters")
```

Εργαστηριακές Ασκήσεις



1. Δεδομένης μιας συμβολοσειράς που σας δίνει ο χρήστης γράψτε τις εντολές που θα:
 - a) Τυπώνει τον πρώτο χαρακτήρα της συμβολοσειράς.
 - b) Τυπώνει το τελευταίο χαρακτήρα.
 - c) Τυπώνει το τελευταίο χαρακτήρα αλλά θα κάνει χρήση της συνάρτησης `len()`.
 - d) Βρίσκει σε ποια θέση της συμβολοσειράς υπάρχει το πρώτο κενό.
 - e) Τυπώνει μόνο την πρώτη λέξη της συμβολοσειράς. Σκεφτείτε ότι οι λέξεις χωρίζονται από κενά. (Να μη χρησιμοποιηθεί η `.split()`).
2. Γράψτε ένα πρόγραμμα Python που θα ζητάει από το χρήστη μια συμβολοσειρά `s` και δύο χαρακτήρες `c1` και `c2`. Το πρόγραμμα θα εκτυπώνει στην οθόνη τη συμβολοσειρά `s`, αλλά όπου εμφανίζεται ο χαρακτήρας `c1` θα εκτυπώνεται ο χαρακτήρας `c2`. Για παράδειγμα, εάν ο χρήστης δώσει τη συμβολοσειρά `Albert Einstein is a scientist` και τα γράμματα `i` και `_`, τότε το πρόγραμμα θα εκτυπώνει στην οθόνη: `Albert E_nste_n_s a sc_ent_st`
3. Μια εταιρεία θέλει να ελέγξει τους κωδικούς των τιμολογίων της. Κάθε κωδικός είναι μια συμβολοσειρά που περιέχει γράμματα και αριθμούς (π.χ. `INV2024`, `sale300`, `A100`). Για να θεωρείται έγκυρος ένας κωδικός πρέπει:
 - Να περιέχει τουλάχιστον 2 ψηφία (αριθμούς)
 - Να περιέχει τουλάχιστον 1 κεφαλαίο γράμμαΝα γράψετε πρόγραμμα που να ζητά τον κωδικό ενός τιμολογίου (μεταβλητή `code`) και να υπολογίζει και να τυπώνει (α) το πλήθος των ψηφίων (β) το πλήθος των κεφαλαίων γραμμάτων, (γ) το πλήθος των μικρών γραμμάτων και (δ) αν το τιμολόγιο είναι έγκυρο ή άκυρο.

1ο Παράδειγμα Εκτέλεσης (με πράσινο η είσοδος του χρήστη)

Δώσε κωδικό τιμολογίου: **INV2024**

Πλήθος ψηφίων: 4

Πλήθος κεφαλαίων γραμμάτων: 3

Πλήθος μικρών γραμμάτων: 0

Το τιμολόγιο είναι: Έγκυρο

2ο Παράδειγμα Εκτέλεσης (με πράσινο η είσοδος του χρήστη)

Δώσε κωδικό τιμολογίου: **sale300**

Πλήθος ψηφίων: 3

Πλήθος κεφαλαίων γραμμάτων: 0

Πλήθος μικρών γραμμάτων: 4

Το τιμολόγιο είναι: Άκυρο

3ο Παράδειγμα Εκτέλεσης (με πράσινο η είσοδος του χρήστη)

Δώσε κωδικό τιμολογίου: **A9b3**

Πλήθος ψηφίων: 2



Πλήθος κεφαλαίων γραμμάτων: 1
Πλήθος μικρών γραμμάτων: 1
Το τιμολόγιο είναι: Έγκυρο

4ο Παράδειγμα Εκτέλεσης (με πράσινο η είσοδος του χρήστη)

Δώσε κωδικό τιμολογίου: **ABc5**
Πλήθος ψηφίων: 1
Πλήθος κεφαλαίων γραμμάτων: 2
Πλήθος μικρών γραμμάτων: 1
Το τιμολόγιο είναι: Άκυρο

4. Η συμβολοσειρά AAAAABCCCCCDDDDCC αποτελείται μόνο από χαρακτήρες του λατινικού αλφαβήτου και το μήκος της είναι 18 χαρακτήρες. Βάσει του μεγέθους της, αν αποθηκευτεί με αυτή τη μορφή, θα χρειαστούν 18 bytes (1 χαρακτήρας = 1 byte). Η χρήση μνήμης μπορεί να περιοριστεί εφαρμόζοντας την εξής μέθοδο συμπίεσης: όταν υπάρχουν περισσότεροι από έναν συνεχόμενοι ίδιοι χαρακτήρες, αυτοί θα διαγράφονται και θα αντικαθίστανται με έναν αριθμό ο οποίος θα υποδεικνύει το πλήθος τους, ακολουθούμενος από ένα αντίγραφο του χαρακτήρα. Για παράδειγμα, η πιο πάνω συμβολοσειρά θα μετατραπεί σε 5AB6C4D2C, η οποία έχει μήκος μόλις 9 χαρακτήρες. Να δημιουργήσετε πρόγραμμα το οποίο δέχεται μια συμβολοσειρά (string) και τυπώνει τη συμπιεσμένη συμβολοσειρά.

Παράδειγμα Εκτέλεσης

Give a string: **AAAAABCCCCCDDDDCC**
Compressed string: 5AB6C4D2C